PowerBuilder Code Examples

Help for the Code Examples application

112 topics · generated 2026-10-27 from the HTML Help (exampl.chm) sources

Contents

1. **PowerBuilder Code Examples**
2. Adding an Other Category to a Graph
3. Attribute Expressions
4. Auto Increment - Call Track
5. Business Class
6. Cancel Retrieval & Printing
7. Change Employee Salary
8. PowerBuilder Code Examples
9. Column Text Value vs. Item Value
10. Computed Fields in Attribute Expr.
11. Context-Sensitive MicroHelp
12. Copy DataWindow
13. Copy, Move, Discard Rows
14. Create From SQL or DWSyntax
15. CurrentRow Function
16. Custom MDI Frame
17. Database Connection
18. Data Explorer
19. Data Manager
20. Data Pipeline
21. DataWindow Buffers
22. DataWindow ExportJson and ImportJson
23. DataWindow Expressions
24. DataWindow Functions
25. DataWindow to HTML Syntax
26. DataWindow Spin Controls
27. DataWindow to Structure Utility
28. DataWindow Validation Rules
29. Date Functions
30. DBError Event
31. DDE Client & Server
32. Department Reports
33. Directory Tree
34. DropDownDataWindow Techniques
35. Dynamically Adding a Series to a Graph
36. Dynamic SQL: 4 Formats
37. Dynamic User Objects
38. Encoding, Hashing and AES Encryption
39. Enhanced Crosstab Reporting
40. Evaluate / ItemError
41. File System Enhancements
42. Function Overloading
43. Garbage Collection
44. GetFirstSheet & GetNextSheet
45. Graphs and Shared DataWindows
46. Hyperlink Controls
47. ImportString() & RowsCopy()
48. Including Blob Columns in a Data Pipeline
49. Item Status & Updating
50. JSONParser and JSONGenerator
51. Mail Merge with RTE-Style DataWindow
52. MAPI Mail - Receive Mail
53. MAPI Mail - Send Mail
54. MenuItem Attributes
55. Metaclass Functionality
56. Modify a DataWindow
57. Multiple Toolbars
58. Nested Reports
59. Newspaper and Sliding Columns
60. OLE 2.0 Control
61. PDF Builder
62. Performance Benchmarks
63. Performance Benchmark Tests
64. PictureListBox Picture Browser
65. Placeholders and OpenURL
66. PopulateError Function
67. PowerBuilt Notepad
68. PowerPuzzle
69. Presentation Styles
70. Printer Enhancements
71. Print Object and Print Screen
72. Progress Bar Controls
73. PSR (Report File) Viewer
74. Query Mode
75. Referencing Nested Reports
76. Registry Editor
77. Registry Functions
78. Reporting with DataWindow Buttons
79. Resizing Controls on a Window
80. RESTClient and HTTPClient
81. Retrieve as Needed & Aggregates
82. RibbonBar Control
83. RichPad RTE Control
84. RowFocusChanging Event

85. Row Selection Techniques
86. Running the Performance Benchmarks
87. Runtime DataWindow Painter
88. Sales Order ListView
89. Sending Email with SMTPClient
90. ShareData With DataStores
91. Share DataWindow buffers
92. Sharing DropDownDataWindows
93. Sort and Filter
94. Stored Procedure Source Data Pipeline
95. Stored Procedure Source DWs
96. System Enhancements
97. Tab Control Phone Directory
98. Table Maintenance with DataWindow Buttons
99. Text Style Changer
100. Towers of Hanoi
101. Trackbar Controls
102. TreeView / DataWindow Linking
103. TreeView Drag/Drop
104. Undeleting a Row
105. Update Trigger
106. User Object Examples
107. Using the Code Examples
108. Various Graph Examples
109. WebBrowser Control
110. Window Scrolling
111. Year 2000 Example
112. Zip and Unzip Files

The Code Example Files

All files required for the Code Examples are in the Examples directory.

PowerBuilder Solution and Libraries

The Code Examples are a PowerBuilder 2025 solution: open `pbexamples.pbsln`. Its project, `PB Examples.pbproj`, lists these libraries. In a solution each library is a folder ending in `.pbl` that holds one source file per object.

- PBEXAMFE.PBL - the application object (examples), the main window and the example list
- PBEXAMD1.PBL - DataWindow objects
- PBEXAMD2.PBL - DataWindow objects
- PBEXAMFN.PBL - global functions
- PBEXAMMN.PBL - menus
- PBEXAMSY.PBL - DataWindow objects that use database-specific SQL (stored procedures, catalog queries), originally written for SQL Anywhere
- PBEXAMUO.PBL - user objects
- PBEXAMW1.PBL - example windows
- PBEXAMW2.PBL - example windows
- PBEXAMW3.PBL - example windows
- PBEXAMNF.PBL - the Modern PowerBuilder examples of features added in PowerBuilder 2017 through 2025 (JSON, REST, encryption, compression, WebBrowser, RibbonBar, PDF Builder and SMTP)

To build the Code Examples executable, deploy the `p_examples_exe` project in PBEXAMFE.PBL. It builds all the libraries into PBDs, which is required because many of the objects in the Code Examples are used dynamically. The `pbexamples_local` and `pbexamples_cloud` projects deploy the examples with PowerClient and PowerServer.

The SVG icons and images the examples use are in the `images` folder. Files the examples no longer use are kept in `_archive`.

PowerBuilder Benchmarks Application

The benchmark example runs a separate PowerBuilder Benchmarks application, built from PBEXBM.PBL as a machine-code executable (EXBM_M.EXE) and a Pcode executable (EXBM_P.EXE). Those files are not included in this repository.

Related Topic

Performance Benchmarks

Adding an Other Category to a Graph

This example shows a graph of the Percentage of Inventory of each product with a quantity greater than 30. All other products (those with quantity less than 30) are lumped together in the Other category.

The number in the upper left corner of the graph is a computed column called other_quantity with the following expression:

```
If( quantity <= 30 , quantity , 0 )
```

The category labels are set as follows:

```
If(quantity <= 30, "Other", description)
```

With the following value:

```
If(quantity <= 30, sum(other_quantity), quantity)
```

This results in the Other category as shown.

Attribute Expressions

Select the region for which you want to allow modifications to the sales orders. All columns that can be modified have a white background. All columns that are protected have a grey background.

Insert Row

Inserts a new sales order for the region that was specified. The region column becomes protected and automatically set to the region that was specified.

Delete Row

Deletes the current sales order, if it is a sales order in the region that can be modified.

Update

Apply changes to the database.

Auto Increment - Call Track

This is an example of using an auto increment column in a table. It is a simple add/update/delete window to maintain data in a table used to track calls.

The id column of the call_track table is set to have a default of AutoIncrement. This allows the database to automatically increment the number whenever a new row is added. There is no need for PowerScript code to determine the call id.

1. Click on any row in the Call List (bottom) DataWindow and see the full information about that row displayed in the top DataWindow. You can update and delete this data as you would with any other DataWindow.
2. To add a new record, click **Add Record**. This opens a blank record in the top DataWindow. Notice that there is no number in the Id column. This value will be calculated by the database when the data is saved with the Save Updates button.

Business Class

This example shows how to separate business rules from the GUI using custom class user objects and inheritance. The business class is created by choosing the Custom Class specification for a new user object. This class (user object) knows how to retrieve, update, insert and delete rows in a DataWindow through its methods (user object functions). This example demonstrates a simple employee object.

The business rules for the employee object are implemented by inheriting from the business class. Any business rules specific to the employee object are defined by overriding the ancestor user object functions. An ancestor update window is used to send retrieve, update, insert, and delete messages to a business object. The employee window, inherited from the ancestor update window, simply specifies that the employee object should receive these messages.

Cancel Retrieval & Printing

Click **Retrieve** to start the DataWindow retrieval. When another dialog opens, click **Cancel** to trigger an event in the main DataWindow that will stop the retrieve.

With data in the DataWindow, try clicking **Print** with the Print Cancel option selected and unselected. Notice the difference of the default cancel dialog popping up when cancels are allowed.

Change Employee Salary

Select a Department, Start Date, and Status and enter a raise or reduction percentage.

Click **Highlight Employees to Change** to highlight the rows in the DataWindow that match the criteria. Use PowerBuilder direct object reference syntax, not **Describe()**, to determine which rows are affected. Also use direct object reference syntax of instead of **Modify()** to change the attributes of the rows.

Click **Calculate Employee Salaries** to change the salary column for those rows. Again, use direct object reference syntax instead of **Describe()** to determine which rows to change. Also use direct object reference syntax instead of **GetItemNumber()** and **SetItem()** to change the salaries. Click **Reset Data** to return the data and DataWindow to their original state.

PowerBuilder Code Examples

Use the PowerBuilder Code Examples to learn many PowerBuilder techniques. You will see examples of common, unique, simple, and advanced PowerBuilder coding.

Related Topics

- Using the Code Examples
- The Code Examples Files

Column Text Value vs. Item Value

Change the data on any column of the DataWindow. Note that as the data is being changed, the text value of the column changes as well.

To apply the text value of the column to the underlying item value of the column, tab to another column or click **AcceptText**. Tabbing to another column causes an `itemchanged` event to occur on the DataWindow, applying the text value (edit field) of the column to the item value. Tab back to the column that was changed, and note that the item value of the column has been changed to the value that you entered.

Clicking **AcceptText** calls the `AcceptText` function, programatically applying the edit field to the underlying item. Note that the item value of the column is modified.

Computed Fields in Attribute Expr.

The minimum and maximum salaries per department are listed at the end of each group.

To locate the employees in each department with minimum and maximum salaries, use the salary column to match up the text colors. If the employee's salary is the minimum salary for the department, the text color of the salary column is blue. If the employee's salary is the maximum salary for the department, the text color of the salary column is red.

Try changing an employee's salary lower than the current minimum salary or higher than the current maximum salary, and observe the text color changes.

Context-Sensitive MicroHelp

Open the Customer or Sales Orders sheets from the **File** menu. Then, right-click the in the sheet and choose **Rows > Insert Row** or **Rows > Retrieve**.

Observe how the MicroHelp at the bottom of the MDI frame window changes, depending on which DataWindow column the cursor is on.

Copy DataWindow

This example shows five different ways to get data into and out of DataWindows. Each test is timed, so that you can see the performance differeces of each method when you run it.

Copy Using GetItem/SetItem

This method uses the appropriate GetItem function (such as `GetItemString()`, `GetItemNumber()`, and `GetItemDate()`) to get the data for each row. It then calls `InsertRow()` to insert a new row into the second DataWindow, and `SetItem()` to set the data.

Copy Using ClipBoard

This method saves the data to the Clipboard using `SaveAs()`, and then imports it into the second DataWindow using `ImportString()`.

Copy Using Structure Array

This method uses the DataWindow object reference syntax to copy all the data into an array of structures that matches the data types of the columns. Then copies the data from there into the second DataWindow.

This is the fastest method to use if you manipulate the data in any way: dump the data into the structure array and loop through it making the necessary changes, then copy it back. This is much faster than stepping through each row of the DataWindow.

DataWindow to Structure Utility example

This example hows how to automatically create a global structure or an autoinstantiated Custom Class NVO that matches the DataWindow.

Copy Using Direct Reference

This method is similar to the previous one, except that it saves one step: the syntax allows you to directly copy all data from one DataWindow to another with one statement, which is faster.

Copy Using RowsCopy()

This method uses the `RowsCopy()` function, which is the fastest way to copy data from one DataWindow to another.

Copy, Move, Discard Rows

Selecting rows

Click a DataWindow row to select it. To select multiple rows, click the starting row; then while pressing and holding the **Shift** key, click the last row.

Moving rows

Drag the row or group of rows to any of the buffers.

Copying rows

While holding down the **Control** key, drag the row or group of rows to any of the buffers.

Discarding rows

Press the **Delete** key to discard rows.

Row counts

Observe the effect of moving, copying, and discarding rows with the displayed row counts. Please refer to the PowerBuilder documentation for the `dwRowsCopy`, `dwRowsMove`, and `dwRowsDiscard` functions to understand how the `Update()` function is affected when moving, copying, or discarding rows

Create From SQL or DWSyntax

Choose a table to show the appropriate columns.

The SQL syntax is built as you click columns or select mutiple columns.

Press the **dwSyntax** to show the DataWindow syntax.

Click the **DataWindow Type** button to change the type of data created.

To create a new DataWindow based on the contents of the Syntax Multiline Edit, click **Create DW**.

CurrentRow Function

The CurrentRow() DataWindow function returns the currently selected row in the DataWindow.

For example, this border expression uses the function in the border and background color attributes of the columns to visibly highlight the current row:

```
If (GetRow() = CurrentRow(), 5, 0)
```

The background color expression is similar:

```
If (GetRow() = CurrentRow(), windowcolor, buttonface)
```

When the window opens, the computed columns, windowcolor and buttonface, are set to be the value of the current window background color and buttonface color.

Custom MDI Frame

Retrieve

Retrieves the DataWindow on the current sheet.

First

Scroll to the first row of the DataWindow on the current sheet.

Next

Scroll to the next row of the DataWindow on the current sheet.

Prior

Scroll to the previous row of the DataWindow on the current sheet.

Last

Scroll to the last row of the DataWindow on the current sheet. The following options will only be available for the Customer List:

New

Inserts a new row in the DataWindow on the current sheet.

Delete

Deletes the current row of the DataWindow on the current sheet.

Apply

Updates the DataWindow on the current sheet.



Database Connection

Click **Retrieve** with different options selected. Notice the time differences between SetTrans and SetTransObject. When the SetTransObject is used, notice that E and **Rollback** are available. This enables you to change a row in the DataWindow, update it, and then roll back or commit the data. If you roll back the data, all of the changes are reversed.

Data Explorer

The Data Explorer is a Windows Explorer-style interface for data manipulation. The explorer consists of two controls linked together: TreeView and a ListView.

The TreeView and ListView controls both display directories in the file system paradigm. The contents of both controls is changed when you select an item in either. The TreeView and ListView controls in this example are populated with data from DataStores. Expand a branch of the TreeView, or double-click an item in the ListView to populate the next level. At the last level (Sales Order items), double-click the item to open an edit window.

Right-click in the ListView to access most of the menu functions.

Menu Options

Open Item
: Populate the next level of data. Expand the branch of the TreeView and display the item's children in the ListView.

Edit Item
: Open a window to edit the item. **Note:** this example does not allow changes to be saved.

Line Up Icons
: This is valid only when the ListView is in Large Icon or Small Icon view.

Data Manager

The Data Manager is an improved Windows Program Manager-style interface for data manipulation. The Data Manager is an Multi-Document Interface (MDI) application with a ListView control on each sheet containing data from the database.

The difference between the Data Manager and the Window Program Manager is that items in the Data Manager can be viewed in any of the four ListView styles (Large Icon, Small Icon, List, and Report). In addition, the Data Manager is used to drill down to other data. This would equate to having groups within groups in Windows. Actually, these differences make the Data Manager more like the Macintosh Finder than the Windows Program Manager.

The ListView controls in this example are populated with data from DataStores. Double-click an item in one ListView to populate the next level. At the last level (Sales Order items), double-click the item to open an edit window. Right-click in the ListView to access most of the menu functions.

Menu Options

Open Item
: Populate the next level of data. Expand the branch of the TreeView and display the item's children in the ListView.

Edit Item
: Open a window to edit the item. **Note**: this example does not allow changes to be saved.

Line Up Icons
: This is only valid when the ListView is in Large Icon or Small Icon view.

Auto Arrange
: This is only valid when the ListView is in Large Icon or Small Icon view.

Minimize on Use
: The Data Manager frame will minimize whenever any item is opened for editing.

Default View
: This is the ListView style that will be used on all new windows which are opened.

Data Pipeline

This example uses the Data Pipeline to:

- Copy a table structure and data
- Copy a table structure, no data
- Execute a transaction - master update

When you open this window or click the Reset Example button, a transaction and master employee tables are created and populated using Data Pipelines. The transaction table is used to update the master table of employees and benefits

Transaction/Master Process

The employee rows in the transaction table are updates that are applied to the master table. Any new employees in the transaction table are added to the master table.

Starting the Pipeline

To follow the process of the Data Pipeline update, select an employee in the transaction DataWindow (on the left side). If the selection is not a new employee, its corresponding row in the master DataWindow (on the right side) is highlighted. To execute the Data Pipeline, selecting the employee's Department and click **Execute**. Notice that the two highlighted rows now match.

Error Handling

If you try to move department 401, the pipeline returns an error, because no department with that id exists in the master table. The Data Pipeline allows you to fix the data in the Pipe Error DataWindow, then click **Repair** to continue the pipeline.

Cancel

Stops execution at any time during the pipeline process.

DataWindow Buffers

This example shows the DataWindow buffers, row and column statuses, and the SQL generated by a DataWindow.

The primary DataWindow is the one displaying the Departments. Try updating the data, deleting rows and filtering it, to observe the effects on the other buffers.

The **Options** menu allows you to change the Update Options for when the data is updated. You can also toggle the SQL Preview.

DataWindow ExportJson and ImportJson

This example converts DataWindow data to JSON and back. The source DataWindow on the left contains embedded employee data, so no database connection is needed; the target DataWindow on the right starts empty.

Export Plain JSON

Calls `ExportJson()`, which produces plain JSON: an array with one object per row, keyed by column name.

Export DataWindow JSON

Calls `ExportJson(True)`, which produces DataWindow JSON. This format also carries the DataWindow identity, the row and column status flags and the original values, so updates can be reproduced elsewhere (for example by a PowerServer Web API).

Changed Rows Only

Calls `ExportJson(Primary!, True, False)` to export only rows that have been modified. Change a value in the source DataWindow first.

ImportJson

Resets the target DataWindow and calls `ImportJson(json, error)` with the JSON shown. Both JSON formats can be imported; JSON keys are matched to column names.

ExportJson and ImportJson were introduced in PowerBuilder 2017 R3.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.



DataWindow Expressions

This example demonstrates the following DataWindow expressions:

```
LIKE
IN
BETWEEN
RGB
CASE
```

This example demonstrates using these expressions in the `text.color` attribute of the columns on the DataWindow:

- Enter a portion of a name (for example, `%ar%`). All names that match change to red.
- Enter a salary range (like `$30,000.00` to `$40,000.00`) All salaries between those numbers change to yellow.
- Enter a list of IDs separated by commas (for example, `1751, 191, 750`). All IDs in the list change to blue.
- Enter numbers between 0 and 255 for the three RGB function values. A CASE expression is used to change the appropriate status to the calculated color.

DataWindow Functions

Undo

Restores the current row/column value back to its original value that was retrieved from the database.

Select Modified

Highlights all rows on the DataWindow that have been modified.

Reset Modified

Resets the updated flags on the DataWindow, so that the deleted and modified counts are reset to zero. Try changing some data on the DataWindow and then clicking the **Reset Modified** button. Then click **Update**. Retrieve the DataWindow again and note that the changes were not made to the database, because the update flags on the DataWindow were reset prior to the update.

DataWindow to HTML Syntax

Select a DataWindow and click **Browse**. The data is saved in an HTML table or form, and displayed in a Web browser. To choose the browser, click the **Select Browser** button.

This example uses the DataWindow `HtmlTable` attribute and `GenerateHTMLForm` function to save the HTML syntax. To use the table syntax, click **Save Rows As > HTML Table for a DataWindow**. **Note**: This action does not work for Nested and Composite reports.

HTML Options

HTML Options Table
: These options change DataWindow attributes , using either the `Modify` function or object reference syntax. **Note**: not all of these options work with all browsers.

Create Style Sheet
: Creates a Cascading Style Sheet (CSS) file that preserves the text styles (such as font face, size, and weight) of the DataWindow, separately from the HTML code.

Wrap Text Within Cells
: Creates fixed-width cells that wrap text within the cells, instead of automatic adjusting cell width.

Table Width
: The width of the entire table (the HTML table width attribute).

Border Width
: The width of the border. `0` indicates no border.

Cell Padding
: The space around the text within each cell.

Cell Spacing
: The space between cells.

Form
: These options are passed as arguments to the `GenerateHTMLForm` function.

Start Row
: The first row of data in the DataWindow to display in the form.

End Row
: The last row of data in the DataWindow to display in the form.

Start Column
: The first DataWindow column to include in the form.

End Column
: The last DataWindow column to include in the form.

Buffer
: The DataWindow buffer from which to retrieve the information.

DataWindow Spin Controls

Click a radio button to choose a date spin control or a numeric spin control. Notice that as the data type of the spin control changes, the displayed attributes for it change as well. Use the DropDownListBox mask to apply different edit masks to the spin control.

Use Code Table

Toggles the code table on/off for the spin control. For example, check Use Code Table and spin up or down on the spin control and notice that it scrolls through its associated code table display values.

Spin Increment

The increment or decrement value for the spin control when the up or down arrows are clicked.

Spin Range Min

The minimum value that can be entered in the spin control.

Spin Range Max

The maximum value that can be entered in the spin control.

DataWindow to Structure Utility

The DataWindow direct object reference syntax allows you to move all the data in the DataWindow with one statement. You can copy data to or from an array of a structure that matches the DataWindow's data structure. This is the fastest way to get data from, or put data into, a DataWindow. See the *Copy DataWindow* example for an example of using this feature.

This utility automatically creates a global structure or an autoinstantiate Custom Class NVO that matches the DataWindow. You can then import the object into a PBL and use it.

Click **Select PBL** to choose the library. Both kinds of library are supported:

- A workspace library: select the `.pbl` file. The DataWindows are listed with `LibraryDirectory` and read with `LibraryExport`.
- A solution library (PowerBuilder 2025 and later), which is a folder ending in `.pbl` that holds one source file per object: select the `.pblmeta` file inside the folder. The DataWindows are listed from the folder's `.srd` files and read from their source.

This is also useful for Distributed PowerBuilder. You cannot pass a DataWindow using Distributed PowerBuilder, which allows you to pass only simple data types to the distributed objects. However, you can pass the data in the DataWindow in a structure array.

DataWindow Validation Rules

This example shows how to call a PowerScript function from the validation rule in a DataWindow. Validating a field in a DataWindow is very easy using the validation selection for a DataWindow field. When the validation rule becomes complex or requires additional functions such as sql calls to a database, you can use a function written in the function painter.

The `dept` field in this example uses a global function as an example of this. In this example, after the department is entered, the function does a lookup in the department table to ensure that it is a valid department.

This example also uses the DataWindow GroupBox.

Date Functions

Choose a day of the week and date that you want to test. Click **Test**. The information displayed shows whether the date and day of the week that you chose match.

DBError Event

Click a radio button to choose how error messages are handled when a database error occurs for the DataWindow.

Click **Retrieve** to retrieve data into the DataWindow.

Enter data into the DataWindow that will cause a database error when the Department table is updated. For example, add a new row, and fill in the Department Name and Manager ID columns, but leave the Department ID blank.

Click **Update**. When the Update function fails, the `dberror` event of the DataWindow is triggered and the appropriate action is taken, based on how you choose to handle error messages. If you choose the default error message, the DataWindow Error message is displayed. The customized message for this example shows the database-specific error code and message that occurs.

DDE Client & Server

Click the start hotlink radio buttons and command buttons. Then open the server window, change the data in the corresponding field, and press **Return**. Try the other radio buttons and command buttons, and observe what happens on the server and client.

Department Reports

Choose a department from the department listing, and then choose a report from the **File** menu. Any number of report instances can be opened. To view reports for a different department, select another department from the department listing and repeat the same technique.

You can also drag a department into an open report sheets. The report changes to that department.

Example

Choose the finance department in the department listing sheet. Open all three reports from the **File** menu. Tile the sheets by clicking **Window > Tile**. Now, drag department names from the department sheet into the open report sheets.

Directory Tree

This example uses recursion to display all of the directories for a drive device in a tree. It demonstrates two ways of presenting the information. First, using a DataWindow to create a tree control, then using the PowerBuilder TreeView control. Use the Tab control to switch between the two views.

Select the drive from the DropDownPictureListBox and click **Scan**. The letter is passed to a recursive function. The function finds all of the subdirectories for that directory, and then calls itself once for each directory. The process repeats until all directories are found.

DropDownDataWindow Techniques

Retrieve and **Insert Row** operate on the main DataWindow.

The DropDownDataWindow (DDDW) used in this example is for the Manager ID column. The contents of the DDDW changes depending on the current row: it lists employees only in the current department.

To change the dataobject of the DDDW, click the **Employee Partial Detail** and **Employee Full Detail** buttons. The Employee Partial Detail DDDW shows only Employee ID, Name, and Department ID. The Full Detail DDDW additionally shows the employee salary and social security number.

Click **Sort** or **Filter** to specify sort or filter criteria for the DDDW.

Example

Click **Retrieve**. Click the Manager ID column of `department 200` and observe that it lists only employees who work in department 200. Changing rows changes the contents of the DDDW. Click **Employee Full Detail** to change the dataobject of the DDDW. Click the Manager ID column again, and notice that the DataWindow changes, now showing the employee's salary and social security number. Click **Sort** and specify a different sort order for the DDDW (such as Employee ID or salary). Click **Filter** and specify filter criteria for the DDDW (such as `salary > 30000`).

Dynamically Adding a Series to a Graph

This example demonstrates how to dynamically add or remove a series in a graph. The first graph is of Revenues per Year. The series added is Revenues per Quarter. The `DataWindow Modify()` function is used to add or remove the series in the clicked event of the button.

Dynamic SQL: 4 Formats

Format 1 Example

Use the DynamicSQL menu to create a new table named Locations in the database. After it is created, the table name appears in the list of tables. Choose **Alter** from the menu to add a fax column to the Locations table. Choose **Drop** from the menu to drop the Locations table (if not, the table will automatically be dropped when the window is closed).

Format 2 Example

This is a simple example of building a dynamic update statement. First, select a department. Then, choose a new manager name and whether to have benefits for the employees in the department. Click **DynamicSQL > Execute** to issue the dynamic UPDATE statement.

Format 3 Example

Choose a department from the ListBox of departments. The employees that work in the current department are listed.

Format 4 Example

1. Enter a dynamic SELECT statement (or keep the default statement provided).
2. Click **DynamicSQL > Describe** to prepare the dynamic staging area with the dynamic SQL statement specified and describe the dynamic staging area into the dynamic description area.
3. Next, enter values for the input parameters. The order of the input parameters that you enter must match the order of the input parameters specified in the dynamic SQL statement.
4. Click **DynamicSQL > Execute** to open a dynamic cursor that is already associated with the dynamic staging area. Each row is fetched using the dynamic description area, and the output parameters of the dynamic description area are shown.

Note

All the formats use MicroHelp to display many of the dynamic SQL statements that are executed.

Dynamic User Objects

Click the first toolbar button to create the Datawindow. Then, click inside the Work Area Window. To remove a button, right-click the area between the button and the border of the User Object.

Encoding, Hashing and AES Encryption

This example uses the native CoderObject and CrypterObject to encode, hash and encrypt text, with no external DLLs or Windows API declarations. Both objects work with blobs, so the text is converted with an explicit encoding: `Blob(text, EncodingUTF8!)`.

Encode / Decode

Shows the text encoded with `Base64Encode`, `Base64UrlEncode`, `HexEncode` and `UrlEncode`, then decodes the Base64 value back to the original text.

Hash / HMAC

Computes `MD5`, `SHA` (SHA1!, SHA256!, SHA512! and SHA3_256!) and an `HMAC` (HMACSHA256!) keyed with the secret key, shown as hexadecimal.

AES-256 Encrypt + Decrypt

Derives a 32-byte key from the passphrase with SHA-256, encrypts the text with `SymmetricEncrypt(AES!, data, key, OperationModeCBC!, iv, PKCSPadding!)`, and decrypts it again with `SymmetricDecrypt`.

The initialization vector (IV) in this example is derived from the passphrase only so that the output is repeatable. In real applications use a random IV for each message and store it with the cipher text.

CoderObject and CrypterObject were introduced in PowerBuilder 2017 R3.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

Enhanced Crosstab Reporting

This example features a crosstab with multiple rows, columns, values.

In the default crosstab, click **Change Crosstab** to change the columns and rows. In the Crosstab dialog box, you can drag columns between the column, row, and value boxes.

The DataWindow has a horizontal split scroll that sets itself to the end of the columns. You can use this control to scroll the right side of the DataWindow while maintaining the titles on the left hand side.

Evaluate / ItemError

Describe(Evaluate), ItemError event, SetActionCode( )

In the salary field, type a mathematical expression. The expression is evaluated when you leave the field.

Example 1

If the salary field currently shows 50,000.00, click the end key to move the cursor to the end of the field and type `*2`. Click **Enter**: the salary is doubled.

Example 2

Move to a new field. Begin typing an expression, such as `(20*(1575+5433))`. Click **Enter**. The expression is evaluated using the correct math precedence.

The item is evaluated in the `ItemError` event, because the math expression is a string and the field is a numeric data type.

Note: The field also has a validation rule that the salary must be greater than zero. If it contains a math expression, the DataWindow field validation is still executed after the math expression is evaluated. This is accomplished by placing the evaluated expression result using `SetText()` in the post event of the `ItemError`.

File System Enhancements

Select an action in the Description column and click **Run** to demonstrate how each of these file system functions can be used.

Function Overloading

PowerBuilder allows function overloading at the object level. This example uses a Global Replace function to show how to overload window functions.

Syntax

The function call used is determined by the values entered. The syntax is displayed at the bottom of the screen. The Global Replace function is called as follows:

`of_global_replace(` *source*, *old*, *new* {, *usecase*} {, *start*, *end*})

Arguments

source
A string containing the text to be changed. Passed by reference.

old
The string to be replaced.

new
The new string to replace it with.

usecase
A boolean indicating whether the replace is case sensitive (optional).

start
An integer indicating where the search begins (optional)

end
An integer indicating where the search ends. The default is `Len(`*source*`)` (optional)

Return Value

Integer. Returns the number of occurrences of old that are replaced by *new*.

Garbage Collection

This example demonstrates the PowerBuilder garbage collection feature. It shows the available memory before and after you create a large number of user objects, with garbage collection turned off and on.

When PowerBuilder returns from executing any system-triggered event, it compares the elapsed time since it last ran a garbage collection pass to the specified garbage collection interval. If the elapsed time exceeds the current garbage collection interval, the garbage collector is run. The default interval is 500 milliseconds (1/2 second).

GetFirstSheet & GetNextSheet

Click **File** and choose one or more of the available sheets (**Window 1** through **Window 5**). After opening one or more sheets, try to open one of them again. An error message says that a window with that title is already open, and lists all of the open sheet titles.

Graphs and Shared DataWindows

The three graphs across the bottom share data with the primary DataWindow at the top. Any changes are shown in the graphs immediately.

The pie chart and the bar chart exclude terminated employees from their graphs. Change the employee status to terminated in a few different departments. Notice how the average age and salary percentage changes. Also note the total count by employee status changes in the column graph.

Changing the employees salary and birth date also changes the graphs.

Hyperlink Controls

This example demonstrates the PictureHyperLink and StaticHyperLink controls.

Click on either the picture or the underlined text to open your Web browser.

ImportString() & RowsCopy()

This example retrieves data in the top DataWindow from the database.

Undo Changes

Make changes in the top DataWindow. When you click **Undo Changes**, the DataWindow is cleared, and `ImportString()` is used to import a copy of the data in the DataWindow as it was when this window opened.

Copy DataWindow

When you click **Copy DataWindow**, the bottom DataWindow is cleared out, and `RowsCopy()` is used to copy the data from the top to the bottom. The bottom DataWindow contains a DropDownDataWindow, which must be retrieved since the parent DataWindow was reset.

Including Blob Columns in a Data Pipeline

You can include Binary Large Object (BLOB) columns in a Data Pipeline. In the Pipeline painter, paint the SQL statement for all other columns first. Then, click **Design/Database Blob** in the menu to add the blob column.

This example creates a copy of the OLE table, which includes a blob column (the OLE object itself).

Click **Execute Pipe** to create and load the new table. As the pipe executes, the number of rows read and written is displayed. When it is done, the DataWindow shows the data from the new table.

This example also includes an OLE 2.0 object in a DataWindow. Double-click any OLE object in the DataWindow to activate it.

Item Status & Updating

Notice that the row status of each row in the DataWindow is shown to the right. The status of the current column is shown at the bottom of the window.

In the DataWindow, you can change the item status for rows and columns in two ways: by clicking **Rows/Insert Row** in the menu to modify data and insert rows, or by explicitly using the `SetItemStatus()` function.

Experiment with item status by changing the data on both original and new rows in the DataWindow. Notice that as you change the item status of a column, the row status changes as well.

You can apply the `SetItemStatus()` function to an individual column or an entire row. To apply `SetItemStatus()` to a column, right-click in the column of the row whose status you want to set, and choose the item status. To set the item status for an entire row, right-click on the row status text (on the right side of the DataWindow) and choose the item status. Options are disabled if they are invalid for the current row status.

To see how item status in a DataWindow affects the Update statement, click **Rows > Update Syntax**, and step throught the generated SQL syntax. The row focus indicator of the DataWindow moves to the row that corresponds to the generated SQL statement. An Update statement is generated for any modifed rows, and an Insert statement is generated for any new rows that were modified. Delete statements are generated for any of the original rows that were deleted.

JSONParser and JSONGenerator

This example shows how to read and write JSON with the native JSONParser and JSONGenerator objects, without parsing strings by hand. A sample JSON document is loaded and parsed when the window opens.

Parse into Tree

Loads the JSON text with `LoadString` and walks it recursively from `GetRootItem`, using `GetItemType`, `GetChildCount`, `GetChildKey` and `GetChildItem`. Each object, array and value is added to the TreeView with its type and value. Edit the JSON and parse it again to see the result; invalid JSON displays the error returned by `LoadString`.

GetItemByPath

Finds a single item directly by its path, such as `/vendor/name` or `/targets/2/type`. Array positions in a path are 1-based.

Generate JSON

Builds a new JSON document in code with JSONGenerator: `CreateJsonObject`, then `AddItemString`, `AddItemNumber`, `AddItemBoolean`, `AddItemDateTime`, `AddItemNull`, `AddItemArray` and `AddItemObject`, and finally `GetJsonString`.

Use as Input

Copies the generated JSON into the input box and parses it, so you can see the round trip.

JSONParser was introduced in PowerBuilder 2017 R2 and JSONGenerator in PowerBuilder 2017 R3.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

Mail Merge with RTE-Style DataWindow

This example uses the Rich Text Edit (RTE) style DataWindow to create a mail merge form letter. The query for this example finds all customers whose last order was before a certain date.

Click the **Retrieve** button to retrieve the data. The < and > buttons go to the previous and next pages, respectively. The toolbar options on the RTE control itself work like any of the popular word processing programs.

MAPI Mail - Receive Mail

Clicking **Read Inbox** reads the contents of your mail inbox into the listing. You can click on a message to see who it is from, whom it is sent to, the subject, and the message. In addition, the DW (DataWindow) flag indicates that this message contains a DataWindow sent from the Send Mail example. Clicking on the flag retrieves the message, rebuilds the DataWindow, and retrieves the data into it. The DataWindow attachment was created using a PSReport format and the `SaveAs()` function.

Deprecated

MAPI mail is deprecated, and the MAPI examples are planned for removal. MAPI needs a MAPI mail client such as classic Outlook for Windows, which Microsoft supports until at least 2029. The new Outlook for Windows does not provide MAPI. PowerBuilder has no built-in replacement for reading a mailbox.

MAPI Mail - Send Mail

This example only works with MAPI mail systems. It constructs a letter and allows you to send it to someone on the mail system. The PowerBuilder example communicates directly with your mail system. It sends a DataWindow attachment in the letter in the format of a PSReport file (PSR) file.

Instructions

Select an address from the mail system and click **Send** to open the mail system's native address selection window. Enter the subject and message, then select a DataWindow. Other options include a receipt upon delivery.

Deprecated

MAPI mail is deprecated, and the MAPI examples are planned for removal. MAPI needs a MAPI mail client such as classic Outlook for Windows, which Microsoft supports until at least 2029. The new Outlook for Windows does not provide MAPI. To send mail from a PowerBuilder application, use the SMTPClient object instead; see Sending Email with SMTPClient.

MenuItem Attributes

Click **Menu 1** or **Menu 2** to change the current menu. The current menu is displayed as three sets of menu items, corresponding to the different levels of the current menu.

Click any level to highlight the menu and show its attributes. Notice that any menu items under selection are listed in the DataWindows on the right. Experiment with changing the attributes of the menu items.

Example

Click **File** in either menu. Its submenu items appear in the second DataWindow. Click the **Exit** menu item in the second DataWindow. Change the enabled, visible, and checked attributes, and use the window's menu to view the changes. To change the MicroHelp of the **Exit** menu item, enter the new MicroHelp and press tab or enter.

Experiment with changing the attributes of the **Exit** toolbar item.



Metaclass Functionality

This example demonstrates the use of metaclass functions to determine whether a function exists on a control before it is called. The example contains several controls on a window, some of which have the cut, copy, and paste functions implemented. When one of those options is selected from the menu, it determines whether or not to call the function based on the current control's class definition.

The metaclass functions, `FindClassDefinition` and `FindMatchingFunction`, are used to get the class definition of the current control and then determine whether it has implemented the desired function.

Modify a DataWindow

This example shows the difference between using `Modify()` and the direct object reference syntax to change the attributes of DataWindow objects. It also highlights the reasons to use each method.

Using Modify()/Describe()

`Describe()` is used to obtain the names of all of the objects in the DataWindow. When an object is selected, `Describe()` is called to obtain the type of object selected. The `Modify()` syntax is built as objects and attributes are selected. Or you can edit it manually.

To build the Modify() syntax:

1. Click on a DataWindow object in the first ListBox, or click on an object in the DataWindow. A partial list of attributes for the object appears in the second ListBox.
2. Choose an attribute from this list to modify its value.
3. If the selected attribute has a set number of values to choose from, the choices appear in the third ListBox, and the current attribute value is selected. To choose a different value, double-click another item in the third ListBox. If the third ListBox does not contain any items, edit the value manually in the Modify syntax field.
4. Click **Apply** to apply the changes to the DataWindow.

Using Direct Object Reference Syntax

The current values of the attributes for the two objects are displayed when the tab page is selected. Direct object reference syntax, not `Describe()`, determines these values. Select new values and click **Apply** to apply the changes to the DataWindow.

Note

Notice that the direct object reference tab page lists only two DataWindow objects and four attributes for each of them. Direct object reference syntax cannot be built dynamically, as `Modify()` and `Describe()` syntax can. You must know the names of the objects and the attributes being modified when you code the syntax. For this reason, `Modify()` and `Describe()` are not obsolete. You still use them to dynamically determine at runtime what attributes of a DataWindow to modify.

Multiple Toolbars

This example demonstrates how to use PowerBuilder toolbar functionality.

Open a sheet and notice that the toolbar sections are all separate toolbars. You can arrange them separately by either dragging them or by clicking **Windows > Toolbars**. The **Edit** menu options (**Cut**, **Copy**, and **Paste**) are in a drop-down toolbar. When you exit, the toolbar positions are saved to either the `example.ini` file or to the registry, and restored the the next time this example is run.

Nested Reports

Opening the reports

Open the various nested reports from the **File** menu. The **File** menu also contains menu items to close or print the active report.

Navigation and display of reports

To view the entire report, choose one of the **Page** commands in the **Display** menu or in the report's context menu. To scale the display area of the report, choose a **Zoom** command in the **Display** menu or in the report's context menu. To display rulers for the report, click **Display > Rulers**.

Newspaper and Sliding Columns

An alphabetical listing of customers in the database in a phone-book format. In the three columns of data, the names are in column order. The items continue from the bottom of the each column to the top of the next column (this is the newspaper column).

The first names slide towards the last names. The last name and first name are not in one field. The first name is a sliding column.

OLE 2.0 Control

This example shows how to:

- Load a file into an OLE control
- Load a row from the EAS Demo DB OLE table into an OLE control
- Create a new object in an OLE control
- Activate the application associated with the data in an OLE control (both in-place and off-site activation)
- Save the contents of an OLE control as a row in the EAS Demo DB OLE table

To display an object in the OLE control:

- To load a file into the OLE control, click **File > Open File**. For example, selecting a BMP or WAV file loads a paintbrush or media player object into the OLE control. An icon in the OLE control indicates if the object has no visual attributes (WAV file).
- To create a new object with no data, click **File > New Object**. This displays a list of OLE-enabled PowerBuilder applications in your configuration. (To add an application to the list, see the PowerBuilder documentation.) Copy the contents of an application to the clipboard, then click **Edit > Paste** to create the object in the application.
- To load an existing OLE object from the database, double-click in the upper right corner of the DataWindow. When the object is loaded, notice that an image of the object is shown. For objects that cannot be shown (such as WAV files), the application icon is shown instead.

When an OLE object is in the OLE control, you can:

- Activate the object by double clicking it. If the object supports OLE 2 in-place activation, the object becomes usable within that control, and the menus become the menu of that application. If the object does not support OLE 2 activation, the application associated with that document is loaded outside of PowerBuilder. If you accidentally load a file whose application is not OLE-enabled, a message is displayed when you try to activate. You might need to use `REGEDIT.EXE` to register your application with Windows. For more information, see the Windows and PowerBuilder documentation.
- Activate the object by selecting **File > Activate** (In-place or Off-site). Again, if the object does not support in-place activation, it loads off-site instead.
- Copy the OLE object to the clipboard by selecting **Edit > Copy**.
- Save the OLE object in the EAS Demo DB OLE table by selecting **File > Save to Database**.

PDF Builder

This example creates PDF files from PowerScript with the PDF Builder objects, and also exports a DataWindow with the built-in NativePDF engine. Files are saved in your TEMP folder, and you are asked whether to open them.

Build PDF with PDF Builder

The of_build_pdf function creates a PDFDocument with a cover page made of PDFText and PDFMultilineText objects (`AddContent`, `AddPage`), then adds the DataWindow with `ImportDataWindow`. It sets the document Title, Author and Subject through `Properties` and saves the file with `Save`.

Protect / Do not allow printing or copying

When checked, the PDF is created with a user password (`Security.UserPassword`) and/or with printing and copying turned off (`Security.AllowPrint` and `Security.AllowCopy`, which require a `Security.MasterPassword`).

DataWindow SaveAs NativePDF

Exports the DataWindow in one step: `Modify("DataWindow.Export.PDF.Method=2")` selects NativePDF, then `SaveAs(file, PDF!, True)` writes the file. No Ghostscript or printer driver is needed.

The PDF Builder objects were introduced in PowerBuilder 2022 and extended in later releases (images, tables of contents, watermarks, attachments and more).

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

Performance Benchmarks

The Performance Benchmarks include 11 tests of PowerBuilder functionality that demonstrate various levels of improvement using machine-code compilation. The source of the application being executed is `pbexbm.pbl`, in the Code Examples directory.

Note about benchmarks: The performance benchmarks included here are intended only as a comparison of p-code and machine-code compilation. Because they are run from the Code Examples, you might not get the same results every time. For the most accurate results, run the two executables (`pbexbm_p.exe` and `pbexbm_m.exe`) individually. You will also get more accurate results if nothing else is running on your computer at the same time, and you disable any network connections.

1. Click **Execute P-Code Benchmarks** to launch `pbexbm_p.exe`, the benchmark application compiled into p-code.
2. Click **Execute Machine-Code Benchmarks** to launch `pbexbm_m.exe`, the benchmark application compiled into Machine-Code.
3. Click **Report >>** to display a report of the results.

Related Topics

- Running the Benchmarks
- The Benchmark Tests

Performance Benchmark Tests

Script Looping

This test is an empty For/Next loop.

The `Iterations` value is the upper limit of the loop. This type of operation shows significant performance improvement with machine-code compilation. Expect around a 2000% increase.

TAK Benchmark

The TAK benchmark is a variant of a benchmark originated by researcher Ikuo Takeuchi, and is widely used to compare programming tools that make extensive use of function calls. The commonly used TAK benchmark algorithm is a variant originated when John McCarthy of Stanford University recalled the original Takeuchi function incorrectly at the start of a benchmarking effort. TAK uses a recursive function that when called with the initial values of 18, 12, and 6, puts the processor through 63,609 function calls and 47,706 subtractions by 1. The `Iterations` value is the number of times to perform the call, passing 18, 12, and 6 each time. This type of operation shows significant performance improvement with machine-code compilation. Expect around a 2000% increase.

Array Sorting

This test uses a bubble-sort algorithm to sort an array of integers. The `Items` value is the size of the array. The items are initially sorted in reverse order to achieve a worst-case sort. This type of operation shows a measurable performance improvement with machine-code compilation. Expect around a 500% increase.

Global Function Overhead

This test calls an empty global function `Iterations` times, and measures the overhead penalty associated with global functions. Function overhead is one of the areas of greatest performance improvement with machine-code compilation. However, the improvement for global functions are not as great as for object functions. Expect around a 1000% increase.

Object Function Overhead

This test calls an empty window function `Iterations` times. This measures the overhead penalty associated with object functions. Function overhead is one of the areas of greatest performance improvement with machine-code compilation. Expect close to a 3000% increase.

Event Overhead

This test calls an empty window event `Iterations` times. This measures the overhead penalty associated with events. There is no difference between object functions and events. Expect results similar to the Object Function Overhead test, close to a 3000% increase.

Window Instantiation

This test opens a window with no controls on it `Iterations` times. This measures the overhead associated with instantiating windows. This is one area where machine-code compilation has no effect. Opening windows and instantiating controls is the same for both types of executables.

String Parsing

This test performs a global change on a string. This includes using the `Pos()` function to find the old text, and the `Left()` and `Right()` functions, together with string concatenation, to make the change. The `Iterations` value is the number of times to perform the global change. This type of operation relies on built-in PowerBuilder functions which are already machine-code compiled. Therefore, you should expect little difference.

Global Variable

This test uses a global variable in a simple integer addition statement. This measures the overhead associated with global variables. The `Iterations` value is the number of times to execute the statement. Global variable overhead is reduced measurably with machine-code compilation. Expect around a 500% increase.

Global Constant

This test uses a global constant in a simple integer addition statement. This measures the overhead associated with constants. The `Iterations` value is the number of times to execute the statement. Global constant overhead is reduced significantly with machine-code compilation. Expect close to a 3000% increase. Also notice that using constants instead of variables results in much greater performance with a machine-code executables than with p-code.

Using DataStores

This test copies all rows from a DataStore into variables using the appropriate GetItem function then writes them to another DataStore using SetItem. The DataWindow used contains data from the sales_order table which has 6 columns and over 600 rows. The `Iterations` value is the number of times to perform the copy. This type of operation shows a small performance improvement with machine-code compilation. Expect around a 50% increase.

PictureListBox Picture Browser

This example demonstrates the use of a PictureListBox. It also shows how to display icons on a window using a ListView control.

Enter a directory name for Picture Directory, or click **Browse** to select one. Under Picture Type, choose **SVG images**, **Bitmaps** or **Icons**, then click **Fill PictureListBox** to list the available pictures.

SVG images are selected when the example opens, and the directory is the examples' `images` folder, which holds the SVG icons the examples use. PictureListBox and Picture controls display SVG files at any size without losing sharpness. Choosing Bitmaps or Icons switches to the examples folder, which contains the older bitmaps and icons.

- To display a picture, select its item in the PictureListBox and click **View Picture**, or double-click the item. The **Picture Zoom** window displays the picture in its original size, or, if you choose **Size to Fit**, the picture changes size when you resize the window.
- To display an icon, select its item in the PictureListBox and click **View Icon**, or double-click the item. The **View Icon** window can display the icon in large (32x32) and small (16x16) sizes.

Placeholders and OpenURL

This example shows two small conveniences: placeholder hints in input fields, and opening links in the default web browser.

Placeholder property

The SingleLineEdit, EditMask and MultiLineEdit controls set the `Placeholder` property. The hint is shown while a field is empty and does not have focus, which can replace a separate label. For an EditMask, placeholders work with string masks only.

Fill In / Clear

Fills the fields with values (the placeholders disappear) or clears them again.

OpenURL buttons

Call `OpenUrl(url)` to open a web page, or a mailto: link that uses the e-mail address entered, in the default browser or mail program. OpenUrl replaces the obsolete Inet.HyperlinkToURL function.

Placeholder support for EditMask and MultiLineEdit was added in PowerBuilder 2025; OpenUrl was added in PowerBuilder 2022 R2.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

PopulateError Function

This example demonstrates the use of the PopulateError function to populate the error object. It asks you to enter some values, validates them, and populates the error object with any error message. It then uses the SignalError function to trigger the SystemError event to display the message.

The PopulateError function sets the error number and error message text properties of the error object. The SignalError function can then be used to trigger the SystemError event. You must code the SystemError event to process the error. The Code Examples SystemError event opens a window that displays the properties of the error object. It includes buttons to exit the application or continue.

PowerBuilt Notepad

To create a new document, click **File > New** in the menu or the **New** toolbar button.

To open a text file, click **File > Open** or the **Open** button.

To delete a file from disk, click **File > Delete**.

To save a document to disk, click **File > Save** or the **Save** button.

To print a document, click **File > Print** or the **Print** button.

The standard text editing and manipulation functions (cut, copy, paste, and so on) are in the **Edit** menu and toolbar buttons. The context menu also has **Edit** commands.

To find and select specific text in the document or jump to a specific line number, click **Search** and choose a command.

Click **Options** to choose various options. The status indicator shows the current text line and column. The clock displays the date and time.

PowerPuzzle

Click **Scramble** to scramble the picture (a landscape with a lake, mountains and a sailboat). Drag and drop each piece to its appropriate place to put it back together. A timer begins when **Scramble** is clicked and the time taken to complete the puzzle is displayed when finished. Click **Show Thumbnail** to view a small version of the finished picture. Click **Solve** if you give up.

Presentation Styles

Each sheet in the MDI frame contains a different presentation style.

These are the basic presentation styles of a DataWindow. In this example, each style is showing the same data in a different presentation style.

The basic result set of the query is: Name, Employee #, Salary, State, and Department.

In PowerBuilder, the DataWindow presentation style can be selected when you first get into the DataWindow painter and select New.

Printer Enhancements

Select an action in the Description column and click **Run** to demonstrate how each of the printer system functions can be used.

Print Object and Print Screen

The objects on the screen are dynamically loaded into the ListBox. The user chooses which objects to print, and each one is sent to a print job using the `Object.Print()` function.

The `PrintScreen()` function prints the entire screen to a print job.

Progress Bar Controls

This example demonstrates horizontal and vertical progress bars. Click the **Start** button to increment the position of the progress bars, then try setting various options in the Options groupbox. To slow down the progress bar, increase the Max Position.

PSR (Report File) Viewer

Click **Select PSR File** to open a file selection dialog box. Choose any `.PSR` file to load it into the DataWindow. PSR files contain both the DataWindow definition and data.
You can treated the loaded PSR file as a normal DataWindow, except that the database connection does not exist.
To create new .PSR files, use the `SaveAs( )` function or menu options with viewing DataWindows.
You can save composite and Nested Report DataWindows as PSR files. The examples directory includes one PSR, `comprep.psr` and `nestrep.psr`, of each type.
In PowerBuilder 8 and later releases, a DataWindow can have buttons with predefined actions . These buttons are also active in a PSR. The examples directory includes two reports that use buttons: `btnmaint.psr` and `btnrept.psr`.

Query Mode

When the upper DataWindow is in Query Mode, you can type criteria for the retrieval in the column fields.

Items placed in the same row use `And` by default. Each additional row of criteria uses `Or` by default.

You can change the defaults by explicitly stating `and/or` in front of the criteria.

As you type the criteria, notice that tabbing out a field adds the criteria to the SQL statement on the bottom. Clicking **Retrieve** also accepts the criteria in the field you are typing in.

QueryMode Examples

ID	State	WHERE Clause
1		Id=1
<1	TX	Id < 1 and State = 'TX'
2 < 9	TX	(id = 2 and State = 'TX') or Id < 9
<10 and > 2		(id < 10 and id > 2)

Referencing Nested Reports

This example shows how to directly reference columns and their data in a nested or composite report.

The report shown is a nested report of all employees and their salaries for each department. Enter a raise percentage for each employee making more than the department average and another percentage for each employee making less. Click **OK** to see the results, including new total salaries for each department.

Use the **>>** button to page forward and the **<<** button to page backward in the report.

Registry Editor

This example is a Windows Registry Editor. It uses a PowerBuilder TreeView and ListView control and Registry functions (`RegistryGet`, `RegistrySet`, `RegistryDelete`, `RegistryKeys`, and `RegistryValues`) to display and update the registry.

It also demonstrates the basics of using a TreeView control. The first level of the tree is added in the window open event. Afterward, each additional level is added when necessary. The code is in the `ItemPopulate` event of the TreeView. `ItemPopulate` is fired once for each item in the tree (the first time it is expanded). Recursion is used to determine the full path to the current item.

This example also demonstrates linking a TreeView control with a ListView control, as used in the Windows Explorer.

Configuration information for the code examples is stored in the registry rather than the `example.ini` file. The branches to this information are opened when the example is started.

Please note: because the registry contains important system settings, exercise extreme caution when updating your registry. It is highly recommended that you confine changes made when running this example to these branches.

Usage

Click on a folder to view the Registry entries and values for that key.

Double-click a folder, or click the + symbol next to it, to view its sub-folders.

Click twice (do not double-click) on a key or entry to change its name, or click **Registry > Rename**.

Double-click an entry or click **Registry > Modify Value** to change the value for an entry. Click **Registry > New > Key** to add a new sub-key under the currently selected key. Click **Registry > New > Value** to add a new entry to the currently selected key. Press Delete (or click **Registry > Delete**) to delete the currently selected key or entry.

Registry Functions

This example demonstrates the use of the PowerBuilder functions to manipulate data in the Windows Registry. It highlights the use of functions to read and write data other than strings.

This example uses the `RegistryGet` and `RegistrySet` functions, overloaded with the data type being read.

Valid types include:

- String
- ExpandString - a string including unexpanded references to environment variables
- Binary
- ULong
- MultiString - an unbounded array of strings

Reporting with DataWindow Buttons

This example is a simple report. The toolbar at the top of the window is built with DataWindow buttons.

The button actions are (left to right): **Retrieve**, **Filter**, **Sort**, **Print Preview**, **Print**, **Save As**, **First Page**, **Previous Page**, **Next Page**, **Last Page**, **Help**, and **Exit**. The processing for all buttons except **Help** and **Exit** is predefined and requires no code in the window. The action of those two buttons is defined as User Defined which results in the `ButtonClicked` event of the DataWindow control being fired when they are clicked. The `DWObject` argument of the event is then checked to determine which button was clicked, and the appropriate action is taken.

Resizing Controls on a Window

Drag the lower right corner of the window to resize it.
Notice that controls like radio buttons and check boxes do not resize properly, because their size is fixed. However, you can resize their text. To resize DataWindows, set the zoom attribute. You can resize all controls by setting their x, y, width, height and fontsize (if the control has a text attribute) attributes; by calculating a new ratio (*new window size/old window size*); and then applying that to the original attributes of the controls. Look at the window function `wf_resize_it( )` to see exactly what happens.

RESTClient and HTTPClient

This example calls a public REST API with the native RESTClient and HTTPClient objects. It uses the free test service at jsonplaceholder.typicode.com, so it requires internet access. You can type another URL to try a different service.

RESTClient.Retrieve

Sends a GET request and loads the JSON response straight into the DataWindow with `Retrieve(dw, url)`. The DataWindow columns (id, name, username, email, phone, website) have the same names as the JSON keys, which is how RESTClient maps the data.

HTTPClient GET

Sends the request with `SendRequest("GET", url)` and shows the status code (`GetResponseStatusCode`), the response headers (`GetResponseHeaders`) and the raw body (`GetResponseBody`).

HTTPClient POST JSON

Builds a JSON body with JSONGenerator, sets the `Content-Type` header and posts it with `SendRequest("POST", url, json)`. The id the server assigns to the new record is read from the response with JSONParser.

RESTClient.SendGetRequest

Shows the simplest way to get a response body as a string with RESTClient.

RESTClient and HTTPClient were introduced in PowerBuilder 2017 R2. Use RESTClient for simple RESTful APIs with flat JSON, and HTTPClient when you need full control of the request or the response is more complex.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

Retrieve as Needed & Aggregates

This example has two DataWindows - both issue the same SQL query. The top DataWindow does not use Retrieve as Needed, while the bottom DataWindow does.

Issue retrieves on both DataWindows by clicking their **Retrieve** buttons. Compare the retrieval times. For accurate comparisons, retrieve both DataWindows several times.

The time taken to retrieve the bottom DataWindow (which uses Retrieve as Needed) includes the time that it takes to first issue an embedded SQL call to count the number of rows that will be retrieved. The top DataWindow (which does not use Retrieve as Needed) does not need an embedded SQL call initially. The row count is calculated in the DataWindow in a computed field. However, notice that using Retrieve as Needed requires much less time to begin displaying the result set to the user.

RibbonBar Control

This example builds a ribbon with the RibbonBar control. A ribbon organizes commands into Categories (tabs), each containing Panels of large buttons, small buttons, check boxes and combo boxes. Each button is bound to a user event of the RibbonBar by name; the event receives the handle of the clicked item, and this example writes the button text to the log.

Build with Script

Removes all categories (`GetCategoryByIndex` and `DeleteCategory`) and builds the ribbon item by item with `InsertCategoryLast`, `InsertPanelLast`, `InsertLargeButtonLast` and `InsertSmallButtonLast`. The click handlers use `GetLargeButton` and `GetSmallButton` to get the clicked item.

Load from JSON

Replaces the ribbon with one described as JSON, using `ImportJSON`. Ribbons can also be loaded from XML or from files.

Show ExportJSON()

Shows the JSON description of the current ribbon, which can be saved and loaded again later.

Ribbons can also be designed visually with the RibbonBar Builder. In PowerBuilder 2025, an existing MDI menu can be displayed as a ribbon without code changes.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

RichPad RTE Control

This example uses the Rich Text Edit (RTE) control to create a simple word processor.

All the expected menu options are available with the following additions:

Send
: This option is on the File menu. It uses MAPI commands to send the Rich Text document as an attachment to a mail message.

Insert Customer Data Field
: This option inserts a data field into the document. The data comes from a DataStore with data from the customer table. This allows mail merge capability.

The toolbar options on the RTE control work like those on many popular word processing programs.

RowFocusChanging Event

The `RowFocusChanging` event is similar to the `RowFocusChanged` event (it fires whenever focus changes from one row to another in a DataWindow) except that it fires before the focus changes and can prevent the focus from changing. The example validates the data entered for the Sales Order items in the `RowFocusChanging` event and if the data fails focus remains on the invalid row.

Validation Rules

- A Product ID must be selected.
- The Ship Date must be greater than or equal to the Order Date.
- The Quantity must be greater than 0.

Row Selection Techniques

To select and highlight a row in the DataWindow, click the row.

To select and highlight a contiguous group of rows, click the first row, then press and hold the **Shift** key while clicking the last row in the group.

To select and highlight a group of multiple rows anywhere in the DataWindow, press and hold **Control**, then clicking the rows you want to select.

To unselect (and unhighlight) one or more rows in a group of selected rows, press and hold **Control**, then click the rows to unselect.

Running the Performance Benchmarks

The title bar displays the type of executable currently being executed.

Click the button for any test to execute it. Except where noted (see *The Benchmark Tests*), iterations refers to the number of times a test is run.

Click **Run all tests** to perform all 11 tests.

Set Machine ID
: Allows you to save information about the machine performing the tests (CPU, speed, memory) which will be displayed on the report.

Save Results
: Saves the results to the Windows Registry.

Report
: Produces a report of the test results.

Related Topic

The Benchmark Tests

Runtime DataWindow Painter

Select a table and columns in the Quick Select Window.

Click **OK** to open a new sheet with your DataWindow populated based on the Quick Select window options.

To create multiple DataWindows, click **Quick Select** again.

To save the DataWindow in a PBL, click **Save Datawindow**.

Sales Order ListView

This example demonstrates the features of the ListView control in an application to add a sales order.

- Enter the sales order information and drag products from the Products window and drop them on the sale order items area.
- Double-click a product to view or edit it.
- Right-click in the Product window to change the view.

Buttons

New Order
: Get the next order number and display a blank screen.

Add Items
: Open the Products window.

Delete Item
: Delete the currently selected item from the sales order.

Update
: Save the order.

Sending Email with SMTPClient

This example sends email directly to an SMTP server with the native SMTPClient and MimeMessage objects. No MAPI client or mail program is required.

Server settings

Set the `Host`, `Port` and `EnableTLS` properties for your server, and `UserName` and `Password` if it requires authentication.

Message

The message is built through the `Message` property (a MimeMessage): `SetSender`, `AddRecipient`, `Subject`, and either `HtmlBody` or `TextBody`, depending on the Send body as HTML option.

Send

Calls `Send()` and shows the result. A negative return value is an SMTPClient error code; see SMTPClient in the PowerBuilder help.

To try the example without a real mail account, run a local test mail server such as smtp4dev or Papercut (for example in a Docker container) and send to localhost.

SMTPClient was introduced in PowerBuilder 2022 R2. It also supports attachments, asynchronous sending (SendAsync) and OAuth 2.0 (XOAUTH2) authentication.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

ShareData With DataStores

This example shows how to use a DataStore to cache data for display in a DataWindow.

Four different DataStores were created and their data retrieved when the window was opened. The first DataStore is shown when the window opens; to select another, click its radio button. `ShareData()` is used to share the data in the DataStore with the DataWindow.

Share DataWindow buffers

The window contains a basic employee DataWindow, with subsidiary DataWindows on the pages of a Tab control for less frequently used information and addresses. Three updateable DataWindows share one primary data source using the ShareData function. Any retrieves made to the primary DataWindow update the secondary DataWindows. Also, any updates to secondary DataWindows update the primary DataWindow store.

Another technique shown is using a Tab control to reduce the clutter of the screen with data and options that are not normally used.

This example also uses the PowerBuilder DataWindow GroupBox.

Sharing DropDownDataWindows

This example uses three DropDownDataWindows:

- Customer
- Sales Representatives
- Financial Codes

The minimized window contains the three DataWindows that are used for the DropDownDataWindows.

To share data between the primary DataWindows on the minimized window with the secondary DropDownDataWindows, click the checkbox. To stop sharing data, click the checkbox again so that it is not checked.

Sort and Filter

Click **Retrieve** to populate the DataWindow.

To change the column on which to sort, choose an item in the DropDownListBox titled, Sort On Column. Choose **Ascending** or **Descending** to set the sort order.

Under Filter Column, choose a or type a column name in the left DropDownListBox, and an operator in the middle control. This makes available in the right control a list of unique values from the DataWindow that match your criteria. Choose a value in the right control to complete the expression. For example, `last name < m'`

The buttons along right side of the window perform basic row functions, like inserting and deleting rows, and updating the database.

Stored Procedure Source Data Pipeline

You can use a Stored Procedure as the source for a Data Pipeline.

This example uses a Data Pipeline to create and load a new table that includes all information about a Sales Order. The table consists of data from several other tables. Click the **Execute Pipe** button to create and load the new table, and display any errors that occur. As the pipe executes, the number of rows read and written is displayed. When it is done, the DataWindow shows the data from the new table.

This example includes both Stored Procedure source and SQL Query source pipelines, so that you can see the difference.

Stored Procedure Source DWs

Choose a customer from the DropDownDataWindow at the top of the window. A graph is displayed, showing the products that the customer has ordered.

Click a product in the graph to display product detail information, and a list of sales orders by the customer for that product. The line item where the product was ordered is highlighted in yellow.

System Enhancements

Select an action in the Description column and click **Run** to demonstrate how each of these system functions can be used.

Tab Control Phone Directory

This example demonstrates two different ways to create a Customer Phone Directory using the PowerBuilder Tab Control:

- Using a single DataWindow and tab pages on the Tab control to filter the DataWindow.
- Using a User Object with a DataWindow as the tab pages on the Tab control and retrieving the data when that object is displayed.

The directory displayed when the window is opened uses the single, floating DataWindow. When you select each letter, the DataWindow is filtered to display only the names starting with that letter.

Click **User Objects** to show the other Tab control. This one consists of tab page user objects (`u_tab_dir`) that each contain a DataWindow with a retrieval argument. When a page is selected, that DataWindow is retrieved. However, the retrieval is only done once. Notice that the pages are displayed much faster when selected a second time.

Click **Change Tabs** to position the tabs somewhere else on the Tab control.

Table Maintenance with DataWindow Buttons

This example is a simple add/update/delete maintenance window for the contact table.

The record scrolling buttons (**First Page**, **Previous Page**, **Next Page** and **Last Page**) as well as the **Add**, **Delete** and **Save** buttons are all predefined. The processing for them is built into the DataWindow. The **Close** and **Help** buttons are User Defined, which means the `ButtonClicked` event of the DataWindow control is fired when they are clicked. The `DWObject` argument of the event is then checked to determine which button was clicked and the appropriate action taken.

Text Style Changer

Ancestor user object

B Change the text to bold

I Change the text to italic

U Underline the text

Inherited user object

B The ancestor script has been overriden. When clicked, instead of making the text bold, this button changes the text to '`bold is not allowed`'. Click the button again to change the text back to its original value.

I The ancestor script has been overriden. The descendant script executes and checks to see if the text is '`please`'. If so, the descendant script dynamically executes the ancestor script to make the text italic.

U The ancestor script has been extended. Therefore, the ancestor script executes first and underlines the text. Then, the descendant script executes and inverts the colors of the text

Towers of Hanoi

The puzzle involves three pegs. The problem is to move all disks from the left peg to the right peg according to certain rules:

Rules

- When you move a disk is moved, you must place it on one of the three pegs.
- You can move only one disk at a time.
- You can move a disk only from the top of the stack (you cannot move a disk that has another disk on top).
- You cannot place a larger disk on top of a smaller one.

Instructions

1. Select the number of disks to be used using the disk edit box. More disks requires more moves to solve problem.\
2. Click **Start** to begin the process. Once it has begun, you must let the problem go to completion.
3. Click **Single Step** to view each move. The moves will stop until you click a button to continue.

Legend of the Towers of Hanoi

The priests in the Temple of Bramah were given a puzzle consisting of a golden platform with three golden needles on which were placed 64 golden disks. The world was to end when they had successfully finished moving the disks, using the rules given above.

Trackbar Controls

This example demonstrates horizontal and vertical trackbars.

Drag the slider on either trackbar to zoom the DataWindow in and out. Try setting various options in the Options groupbox and see how it affects the trackbar controls.

Options

Line Size
How far the slider moves in response to keyboard input from the arrow keys.

Page Size
How far the slider moves in response to keyboard input from the PageUp or PageDown keys or the mouse. When PageSize is set to 1 the slider moves 1 increment in the range of values

Start Position
Where the slider is when you open the window.

Min Position
The beginning of the range of values.

Max Position
The end of the range of values.

Slider Size
The size of the slider and the width of the trackbar.

Tick Frequency
How frequently tick marks display.

Tick Position
Where the tick marks display.

Slider
Unselect to hide the slider.

TreeView / DataWindow Linking

This example demonstrates using a TreeView control for data navigation and DataWindows to update the data.
The TreeView control is populated with data from DataStores. Each level of the TreeView uses a different DataStore. The level is populated the first time an item is expanded.
The DataWindows on the right of the screen show all records in the path down to the currently selected one. Whenever a DataWindow gets focus, its corresponding TreeView item is selected.
A state picture is used to display a disk icon next to any TreeView item that is changed. This picture is hidden when the data is saved or the changes are canceled.

TreeView Drag/Drop

This example demonstrates how to update a TreeView by dragging and dropping branches.

1. In the salary field, type a mathematical expression.
2. Drag an employee from one department to another to perform the transfer. You can drop the employee on the new department or on any employee in the new department. As you drag, notice that the current drag target (the new department) is highlighted. When the employee is dropped, it is inserted in the proper place in the department based on alphabetical order.
3. Click **Update** to save the changes, or **Cancel** to cancel them.

Undeleting a Row

Retrieve

Retrieves data from the database into the DataWindow.

Update Row

Updates the database with any changes that were made in the DataWindow.

Delete Row

Deletes the current row from the DataWindow.

UnDelete Row

Undeletes the last row that was deleted from the DataWindow.

Update Trigger

Change the Manager ID of a department.

Click **Update**. A trigger is fired that updates the employee table. Notice that when the update is complete, the data in the employee table is also updated to correspond to the changes that were made to the department table.

User Object Examples

This window contains six user objects. Each user object encapsulates its functionality, so using them in a window requires little coding.

Enter a valid Friday
: Checks to see if the date entered is a Friday. If not, sounds a beep and clears the date.

Enter a valid numeric
: Allows only numeric characters to be entered.

Overtype mode SLE
: This SingleLineEdit user object allows characters to be inserted in overtype mode, instead of the standard insert mode.

Numeric Range Checker
: Allows numbers to be entered only within the specified range. If the number is not in the specified range, an error message is returned.

Date Scroller
: Scrolls the entire date by one day. With standard spin controls, only the portion of the date that is being edited scrolls.

Progress Meter
: Updates the width of the status bar and changes the percentage complete, using a user object function.

Using the Code Examples

The PowerBuilder Code Examples front end is divided into two main parts: The left side of the screen includes the list of all examples, while the right side of the screen displays information about the currently selected example.

Examples List

The Code Examples window lists available examples in a TreeView. Examples are grouped into categories, like DataWindows and Functions. Examples can appear in more than one category. In addition to the main Examples page, the Tab control has a Search page.

Showing New and Changed Examples

In the Examples page, you can view all examples, or use the DropDownPictureListBox to filter examples by their status. Each example is listed with one of these status symbols:



Examples marked with this symbol are part of the original set of code examples and have been changed only where necessary to replace obsolete syntax.



Examples marked with this symbol have been changed significantly in subsequent releases.



Newer examples are marked with this symbol.

Choose **Modern (2017+)** in the list to show only the examples of features added in PowerBuilder 2017 and later. These examples are also grouped under the Modern PowerBuilder category, and none of them require the demo database.

Searching the Code Examples

The second tab page is used to search for examples. You can search for examples that:

- Use one or more specified events
- Use one or more specified PowerScript functions
- Use one or more specified user-defined functions
- Reference one or more specified objects

Data Explorer Display

When you select an example in the TreeView, information about the example is displayed in the Data Explorer on the right side of the window. Data Explorer has two tab controls:

Description/Techniques
: Describes the functionality and techniques used in the example.

Related Objects

Contains a TreeView with cross-reference and metaclass information all the objects used in the example. You can navigate the tree to nested objects and all scripts. In a script, double-click any event or function to open a window and view its code.

Menu Items

You can access many of the menu commands by right-clicking items in the Examples list.

File

Example Report

Shows a printable report of all the example information for the currently selected example.

Go To Topic

Scrolls to the first example of the selected topic.

Change Database Connection

Opens a window where you can change the connection information for the EAS Demo database.

Run!

Runs the selected example. You can also click **Run!** in the menu, or double-click an example in the list to run it.

Help

Contents

Opens the topic, *PowerBuilder Code Examples*.

Using the Code Examples

Opens this help topic.

Current Example Help

Opens help for the currently selected example.

About Displays information about the Code Examples.

Note

Configuration information for the Code Examples is stored in the registry rather than the `example.ini` file. It is stored under `HKEY_CURRENT_USER\Software\Appeon\PowerBuilder\`*`version`*`\Code Examples`. You can view this information with the Registry Editor example included.

Various Graph Examples

On the Stat sheets, try changing the values that are graphed. Try graphing multiple values at the same time.

Change different aspects of the graph using **Style** menu commands.

You can copy the graphs to the clipboard using **Edit** menu picks.

WebBrowser Control

This example uses the WebBrowser control, which embeds a modern Chromium-based browser (Microsoft Edge WebView2), and shows how PowerScript and JavaScript in the page can call each other. The Microsoft Edge WebView2 runtime must be installed.

Go, Back, Fwd

Navigate with `Navigate`, `GoBack` and `GoForward`. The TitleTextChanged event writes each page title to the log.

Demo

Loads an HTML page from a string with `NavigateToString`. Its button calls `window.webBrowser.ue_from_js(message)` in JavaScript, which runs the ue_from_js user event of the control in PowerScript; the string the event returns is shown in the page. The event is registered with `RegisterEvent("ue_from_js")` in the NavigationStart event, after the browser has finished initializing.

EvaluateJavascriptSync

Runs the JavaScript in the text box in the current page and logs the result, which is returned as a JSON string containing the type and value. After clicking Demo, the script changes text in the page.

A user event called from JavaScript must not call Navigate or EvaluateJavascriptSync itself.

The WebBrowser control was introduced in PowerBuilder 2019 R2.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.

Window Scrolling

Scroll Lines Per Page

This value determines the number of lines on a page for the window. It affects the number of PowerBuilder units the window scrolls when you click inside the vertical scroll bar.

Units Per Scroll Line

This value determines the number of PowerBuilder units that the window scrolls when you click the up or down arrow of the vertical scroll bar. The default value (zero) scrolls the window 1/100th of the window height when you click the arrows.

Example

Try decreasing the scroll units per line, and click **Apply** for your changes to take effect. Click and hold the up or down arrow on the scroll bar, and notice that the window scrolls a lesser amount. Increase the scroll units per line, and notice that the window scrolls a greater amount.

Year 2000 Example

This example demonstrates how PowerBuilder functionality works with dates before and after the year 2000. It includes examples of date manipulation in both DataWindows and windows.

Both two- and four-digit dates are entered, sorted, and converted. Two-digit dates with years of 00-49 are given a century of 20, while two-digit dates with years of 50-99 are given a century of 19.

Zip and Unzip Files

This example creates and reads archives with the native CompressorObject and ExtractorObject. When the window opens, a few of the Code Examples files are added to the list and the archive name is set to a file in your TEMP folder (found with the Keyword context service).

Add Files / Clear List
: Selects the files to compress.

Compress to ZIP
: Calls `Compress(files[], archive, ArchiveFormatZIP!)`. If you enter a password, the `Password` property is set first and the archive is encrypted with AES-256.

List Contents
: Lists the files in the archive with `ExtractorObject.GetFilesList`. Enter the same password for an encrypted archive.

Extract To Folder
: Extracts the whole archive to a folder you select with `Extract(archive, folder)`.

Compress a Blob with GZIP and Back
: Compresses a blob in memory with `Compress(blob, ref blob, ArchiveFormatGZip!)` and restores it with `Extract(blob, ref blob, ArchiveFormatGZip!)`, without touching the file system, and reports the sizes.

CompressorObject and ExtractorObject were introduced in PowerBuilder 2019. They also support 7Z, GZIP and TAR archives.

This example is in PBEXAMNF.PBL, with the other Modern PowerBuilder examples.